

JavaScript Intermediate — Closures & Lexical Scope

Closures JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ intermediate concepts-এর একটি।

Closure বুঝতে পারলে আমরা শুধু closure কী সেটা জানব না—বরং বুঝতে পারব:

- JavaScript কীভাবে scope খুঁজে
- একটা function কীভাবে তার বাইরের variable মনে রাখে
- Function return হয়ে যাওয়ার পরও variable কীভাবে accessible থাকে
- Private state কীভাবে তৈরি করা যায়
- Memoization কীভাবে কাজ করে
- Partial application কী
- Variable shadowing কীভাবে scope resolution-কে প্রভাবিত করে
- React-এর অনেক pattern-এর পেছনে closure কেন গুরুত্বপূর্ণ

Module 1 — Lexical Scope & Closures

1. Scope কী?

Scope হলো কোনো variable কোথা থেকে accessible হবে তার boundary।

সহজভাবে:

Scope ঠিক করে দেয় — কোন জায়গা থেকে কোন variable-কে access করা যাবে।

Example:

```
const name = "Sharif";

function greet() {
  console.log(name);
}

greet();
```

এখানে `name` function-এর ভিতরে declare করা হয়নি।

কিন্তু `greet()` `name` access করতে পারছে।

কারণ `name` outer/global scope-এ আছে।

2. Lexical Scope কী?

JavaScript-এর scope মূলত **lexical/static scope**।

এর অর্থ:

Function কোথায় লেখা হয়েছে তার উপর scope নির্ভর করে, function কোথা থেকে call করা হয়েছে তার উপর নয়।

Example:

```
const name = "Shariar";

function outer() {
  const message = "Hello";

  function inner() {
    console.log(message);
  }

  inner();
}

outer();
```

`inner()` function-এর ভিতরে `message` নেই।

তাই JavaScript outer lexical environment-এ খুঁজবে।

```
inner()
  ↓
inner scope
  ↓
outer scope
  ↓
global scope
```

এটাই **scope chain**।

3. Scope Chain

যখন JavaScript কোনো variable দেখে, তখন প্রথমে current scope-এ খোঁজে।

না পেলে outer scope-এ যায়।

তারপর তার outer scope-এ যায়।

Example:

```
const a = 10;

function outer() {
  const b = 20;

  function inner() {
    const c = 30;

    console.log(a);
    console.log(b);
    console.log(c);
  }

  inner();
}

outer();
```

`inner()` তিনটা variable-ই access করতে পারে।

কারণ:

```
inner scope
├── c
│
↓
outer scope
├── b
│
↓
global scope
├── a
```

4. Nested Function

Closure বোঝার জন্য nested function খুব গুরুত্বপূর্ণ।

```
function outer() {  
  const message = "Hello";  
  
  function inner() {  
    console.log(message);  
  }  
  
  inner();  
}  
  
outer();
```

এখানে `inner()` হলো `outer()`-এর ভিতরের function।

`inner()` তার parent scope-এর variable access করতে পারে।

5. এখন Closure কী?

Closure হলো এমন একটি function যেটা তার **lexical environment**-এর **reference** মনে রাখে, এমনকি outer function execution শেষ হয়ে যাওয়ার পরেও।

এই definition একটু কঠিন।

চলো example দেখি।

```
function outer() {  
  const message = "Hello";  
  
  function inner() {  
    console.log(message);  
  }  
  
  return inner;  
}  
  
const greet = outer();
```

```
greet();
```

Output:

```
Hello
```

এখানে গুরুত্বপূর্ণ বিষয় হলো:

```
const greet = outer();
```

`outer()` execution শেষ হয়ে গেছে।

সাধারণভাবে মনে হতে পারে `message` আর থাকার কথা না।

কিন্তু:

```
greet();
```

করলে এখনও:

```
Hello
```

পাওয়া যাচ্ছে।

কেন?

কারণ `inner()` তার outer lexical environment-এর `message` -কে remember করে রেখেছে।

এটাই **closure**।

6. Closure-এর Mental Model

এই codeটা দেখো:

```
function outer() {  
  let count = 0;
```

```

    return function inner() {
      count++;
      console.log(count);
    };
  }

  const counter = outer();

  counter();
  counter();
  counter();

```

Output:

```

1
2
3

```

এখন প্রশ্ন:

`outer()` তো একবারই execute হয়েছে।

তাহলে `count` কীভাবে $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ হলো?

কারণ `counter` function-এর সাথে তার lexical environment-ও connected আছে।

Mental model:

```

outer()
├──
├── count = 0
└── inner()
    └── remembers count

```

তারপর:

```

counter()
  ↓
count = 1

counter()

```

```
    ↓  
count = 2  
  
counter()  
    ↓  
count = 3
```

7. Closure তৈরি হওয়ার জন্য সাধারণত কী লাগে?

সাধারণ pattern:

```
function outer() {  
  let variable = value;  
  
  return function inner() {  
    // uses variable  
  };  
}
```

অর্থাৎ:

1. Outer function আছে
2. তার ভিতরে variable আছে
3. Inner function সেই variable ব্যবহার করছে
4. Inner function বাইরে return করা হচ্ছে / কোথাও retained হচ্ছে

8. Closure কি শুধু return করা function-এর ক্ষেত্রেই হয়?

না।

Closure শুধু `return` করার কারণে তৈরি হয় না।

যেকোনো function তার lexical scope-এর variable access করতে পারে।

Example:

```
function setup() {  
  const message = "Hello";  
  
  setTimeout(() => {
```

```
        console.log(message);
    }, 1000);
}

setup();
```

`setTimeout` callback `message` access করছে।

এটাও closure-এর example।

9. Practical Use Cases of Closures

Closures বাস্তবে অনেক জায়গায় ব্যবহৃত হয়।

Common use cases:

- Private state
- Data encapsulation
- Factory functions
- Memoization
- Partial application
- Function factories
- Event handlers
- Callbacks
- Timers
- Module pattern
- React event handlers/hooks-এর কিছু behavior বোঝা

10. Closure দিয়ে Function Factory

```
function createMultiplier(multiplier) {
  return function (number) {
    return number * multiplier;
  };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(5));
console.log(triple(5));
```

Output:

```
10  
15
```

এখানে:

```
double
```

এর closure-এ:

```
multiplier = 2
```

আর:

```
triple
```

এর closure-এ:

```
multiplier = 3
```

প্রতিটা function নিজের আলাদা environment মনে রাখে।

11. একই Outer Function থেকে Multiple Closures

```
function createCounter() {  
  let count = 0;  
  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const counter1 = createCounter();  
const counter2 = createCounter();  
  
console.log(counter1());
```

```
console.log(counter1());  
  
console.log(counter2());  
console.log(counter2());
```

Output:

```
1  
2  
1  
2
```

কারণ:

```
counter1 → নিজের count  
counter2 → নিজের count
```

দুইটা closure-এর environment আলাদা।

Module 2 — Closure-Based Private State & Module Pattern

1. JavaScript-এ Private State কী?

Private state বলতে এমন data বোঝায় যেটা সরাসরি বাইরে থেকে access বা modify করা যাবে না।

ধরো:

```
const account = {  
  balance: 1000  
};  
  
account.balance = 999999;
```

এখানে বাইরে থেকে `balance` পরিবর্তন করা যাচ্ছে।

Closure দিয়ে আমরা data encapsulate করতে পারি।

2. Closure দিয়ে Private Variable

```
function createBankAccount() {  
  let balance = 0;  
  
  return {  
    deposit(amount) {  
      balance += amount;  
    },  
  
    withdraw(amount) {  
      if (amount <= balance) {  
        balance -= amount;  
      }  
    },  
  
    getBalance() {  
      return balance;  
    }  
  };  
}  
  
const account = createBankAccount();  
  
account.deposit(1000);  
  
console.log(account.getBalance());
```

Output:

1000

কিন্তু:

account.balance

হলে:

undefined

কারণ `balance` object-এর property নয়।

এটা closure-এর private variable ।

3. Data Encapsulation

এই pattern-এ:

```
Private data
  ↓
closure
  ↓
public methods
```

শুধু methods-এর মাধ্যমে data access করা যায় ।

Example:

```
function createUser(name) {
  let password = "secret123";

  return {
    getName() {
      return name;
    },

    checkPassword(input) {
      return input === password;
    }
  };
}
```

এখানে:

```
password
```

private ।

Outside থেকে:

```
user.password
```

access করা যাবে না।

4. Module Pattern

Closure ব্যবহার করে JavaScript-এ আগে খুব popular ছিল **Module Pattern**।

```
const counterModule = (function () {  
  let count = 0;  
  
  function increment() {  
    count++;  
  }  
  
  function decrement() {  
    count--;  
  }  
  
  function getCount() {  
    return count;  
  }  
  
  return {  
    increment,  
    decrement,  
    getCount  
  };  
})();
```

Use:

```
counterModule.increment();  
counterModule.increment();  
  
console.log(counterModule.getCount());
```

Output:

2

কিছু:

```
counterModule.count
```

হবে:

```
undefined
```

কারণ `count` private ।

5. Modern JavaScript-এ Module Pattern

Modern JavaScript-এ ES Modules আসার কারণে traditional IIFE module pattern-এর প্রয়োজন অনেক কমেছে ।

আমরা এখন ব্যবহার করি:

```
export  
import
```

Example:

```
// counter.js  
  
let count = 0;  
  
export function increment() {  
  count++;  
}  
  
export function getCount() {  
  return count;  
}
```

অন্য file:

```
import { increment, getCount } from "./counter.js";
```

তবুও closure জানা গুরুত্বপূর্ণ কারণ private state, factories এবং functional patterns-এ closure এখনও ব্যাপকভাবে ব্যবহৃত হয় ।

6. Factory Function + Private State

```
function createUser(name) {  
  let loginCount = 0;  
  
  return {  
    login() {  
      loginCount++;  
      console.log(`${name} logged in`);  
    },  
  
    getLoginCount() {  
      return loginCount;  
    }  
  };  
}  
  
const user = createUser("Shariar");  
  
user.login();  
user.login();  
  
console.log(user.getLoginCount());
```

Output:

```
Shariar logged in  
Shariar logged in  
2
```

Module 3 — Memoization with Closures

1. Memoization কী?

Memoization হলো:

Expensive function-এর previous result cache করে রাখা, যাতে একই input আবার এলে calculation পুনরায় করতে না হয়।

Example:

```
function square(number) {  
  console.log("Calculating...");  
  
  return number * number;  
}
```

বারবার:

```
square(5);  
square(5);  
square(5);
```

করলে calculation তিনবার হবে।

Memoization করলে প্রথমবার calculation হবে।

পরেরবার cached result পাওয়া যাবে।

2. Closure কেন Memoization-এর জন্য useful?

আমরা cache-কে private রাখতে পারি।

```
function memoizeSquare() {  
  const cache = {};  
  
  return function (number) {  
    if (number in cache) {  
      console.log("From cache");  
      return cache[number];  
    }  
  
    console.log("Calculating");  
  
    const result = number * number;  
  
    cache[number] = result;  
  
    return result;  
  };  
}
```

Use:

```
const square = memoizeSquare();

console.log(square(5));
console.log(square(5));
console.log(square(10));
console.log(square(10));
```

প্রথম 5:

```
Calculating
25
```

দ্বিতীয় 5:

```
From cache
25
```

3. এখানে Closure কোথায়?

এই:

```
const cache = {};
```

`memoizeSquare()` execution শেষ হওয়ার পরও retained থাকে।

কারণ returned function:

```
return function (number) {
  ...
};
```

`cache` ব্যবহার করছে।

অর্থাৎ:

```
square function
  ↓
closure
```

↓
cache

4. Generic Memoization Function

আমরা যেকোনো function memoize করতে পারি।

```
function memoize(fn) {  
  const cache = new Map();  
  
  return function (...args) {  
    const key = JSON.stringify(args);  
  
    if (cache.has(key)) {  
      return cache.get(key);  
    }  
  
    const result = fn(...args);  
  
    cache.set(key, result);  
  
    return result;  
  };  
}
```

Example:

```
function add(a, b) {  
  console.log("Calculating...");  
  return a + b;  
}  
  
const memoizedAdd = memoize(add);  
  
console.log(memoizedAdd(10, 20));  
console.log(memoizedAdd(10, 20));
```

প্রথমবার calculation হবে।

দ্বিতীয়বার cached result আসবে।

5. Memoization কখন useful?

Useful যখন:

- একই input বারবার আসে
- calculation expensive
- function pure
- result input-এর উপর নির্ভরশীল

Example:

```
f(10) → expensive calculation  
f(10) → cached result  
f(10) → cached result
```

6. Memoization-এর জন্য Function Pure হওয়া ভালো

Pure function:

```
function add(a, b) {  
  return a + b;  
}
```

একই input → একই output ।

কিন্তু:

```
function getTime() {  
  return Date.now();  
}
```

এটা memoize করলে সমস্যা হতে পারে ।

কারণ একই input না থাকলেও result সময়ের সাথে পরিবর্তিত হয় ।

Module 4 — Partial Application with Closures

1. Partial Application কী?

Partial application হলো:

একটি function-এর কিছু argument আগে থেকেই set করে নতুন function তৈরি করা।

ধরো:

```
function add(a, b, c) {  
  return a + b + c;  
}
```

আমরা চাই:

```
a = 10
```

আগেই fixed থাকুক।

তাহলে নতুন function:

```
addWith10(b, c)
```

তৈরি করতে পারি।

2. Closure দিয়ে Partial Application

```
function add(a, b, c) {  
  return a + b + c;  
}  
  
function partialAdd(a) {  
  return function (b, c) {  
    return add(a, b, c);  
  };  
}  
  
const add10 = partialAdd(10);
```

```
console.log(add10(20, 30));
```

Output:

```
60
```

এখানে:

```
a = 10
```

closure-এর মধ্যে remember হচ্ছে।

3. Real-World Example

ধরো discount calculation:

```
function calculatePrice(discount, price) {  
  return price - price * discount;  
}
```

আমরা যদি 20% discount-এর জন্য function বানাতে চাই:

```
function createDiscount(discount) {  
  return function (price) {  
    return price - price * discount;  
  };  
}
```

তারপর:

```
const twentyPercentOff = createDiscount(0.2);  
  
console.log(twentyPercentOff(1000));  
console.log(twentyPercentOff(500));
```

Output:

```
800  
400
```

এখানে:

```
discount = 0.2
```

closure-এর মাধ্যমে retained ।

4. Partial Application vs Currying

দুটো related হলেও একই জিনিস নয় ।

Partial Application

কিছু argument আগে থেকে set করা:

```
function add(a, b, c) {  
  return a + b + c;  
}  
  
const add10 = partialAdd(10);
```

Currying

একবারে একটি করে argument নেওয়া:

```
function add(a) {  
  return function (b) {  
    return function (c) {  
      return a + b + c;  
    };  
  };  
}
```

Use:

```
add(10)(20)(30);
```

Result:

60

5. Generic Partial Application

```
function partial(fn, ...fixedArgs) {  
  return function (...remainingArgs) {  
    return fn(...fixedArgs, ...remainingArgs);  
  };  
}
```

Example:

```
function multiply(a, b, c) {  
  return a * b * c;  
}  
  
const multiplyBy2 = partial(multiply, 2);  
  
console.log(multiplyBy2(3, 4));
```

Output:

24

কারণ:

```
multiply(2, 3, 4)  
= 24
```

Module 5 — Variable Shadowing

1. Shadowing কী?

যখন inner scope-এ একই নামের variable declare করা হয় এবং সেটা outer variable-কে temporarily hide করে, তখন তাকে **variable shadowing** বলে।

Example:

```
const name = "Global";

function greet() {
  const name = "Local";

  console.log(name);
}

greet();
```

Output:

Local

Outer:

```
const name = "Global";
```

থাকলেও inner `name` সেটাকে shadow করছে।

2. Scope Chain + Shadowing

```
const name = "Global";

function outer() {
  const name = "Outer";

  function inner() {
    const name = "Inner";

    console.log(name);
  }
}
```

```
    }  
    inner();  
  }  
  outer();
```

Output:

Inner

JavaScript প্রথমে current scope-এ খুঁজে।

inner → name found

তাই আর outer scope-এ যেতে হয় না।

3. Shadowing Mental Model

```
Global  
└─ name = "Global"  
  
    Outer  
    └─ name = "Outer"  
  
        Inner  
        └─ name = "Inner"
```

`inner()` যখন:

```
console.log(name);
```

দেখে:

1. Inner scope → পাওয়া গেছে

তাই search stop।

4. Block Scope Shadowing

`let` এবং `const` block-scoped।

```
let message = "Outside";

{
  let message = "Inside";

  console.log(message);
}

console.log(message);
```

Output:

```
Inside
Outside
```

5. `var` এবং Shadowing

`var` function-scoped।

```
var name = "Global";

function test() {
  var name = "Function";

  console.log(name);
}

test();

console.log(name);
```

Output:

Function
Global

এখানে function scope-এর `name` global `name` -কে shadow করছে।

6. Shadowing with Function Parameters

Function parameter-ও local variable-এর মতো scope তৈরি করে।

```
const name = "Global";

function greet(name) {
  console.log(name);
}

greet("Shariar");
```

Output:

Shariar

Parameter:

name

global `name` -কে shadow করছে।

7. Illegal Shadowing

`let` / `const` এবং `var` নিয়ে কিছু shadowing restriction আছে।

এই codeটি সমস্যা তৈরি করবে:

```
let name = "Global";

{
```

```
var name = "Block";
}
```

কারণ `var` block scope মানে না।

তাই একই scope-এর conflict তৈরি হয়।

তবে nested function scope আলাদা:

```
let name = "Global";

function test() {
  var name = "Function";

  console.log(name);
}
```

এটা valid।

8. Shadowing vs Reassignment

দুটো এক জিনিস নয়।

Reassignment

```
let count = 10;

count = 20;
```

একই variable-এর value পরিবর্তন হয়েছে।

Shadowing

```
let count = 10;

{
  let count = 20;
}
```

এখানে নতুন variable তৈরি হয়েছে।

Outer `count` পরিবর্তন হয়নি।

9. Closure + Shadowing একসাথে

এই exampleটা খুব গুরুত্বপূর্ণ:

```
const message = "Global";

function outer() {
  const message = "Outer";

  return function inner() {
    console.log(message);
  };
}

const fn = outer();

fn();
```

Output:

Outer

কেন?

`inner()` lexical scope অনুযায়ী `outer()` -এর `message` -কে capture করেছে।

Global `message` shadowed হয়েছে।

Mental model:

```
Global
└─ message = "Global"

Outer
└─ message = "Outer"
    └─ Inner
        └─ uses "Outer"
```

10. Closure + Loop — Important Interview Concept

Closure বোঝার সবচেয়ে famous example:

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 1000);  
}
```

Output:

```
3  
3  
3
```

কেন?

`var` function-scoped এবং তিনটি callback একই `i` binding share করে।

Loop শেষ হওয়ার সময়:

```
i = 3
```

তখন callbacks execute করে।

`let` ব্যবহার করলে

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 1000);  
}
```

Output:

```
0  
1  
2
```

কারণ `let` loop-এর প্রতিটি iteration-এর জন্য আলাদা binding তৈরি করে।

Mental model:

```
Iteration 1 → i = 0  
Iteration 2 → i = 1  
Iteration 3 → i = 2
```

প্রতিটি callback তার corresponding `i` পায়।

11. Closure কি Memory Leak তৈরি করতে পারে?

Closure নিজে memory leak নয়।

কিন্তু closure কোনো বড় object বা DOM reference ধরে রাখলে এবং সেটা আর প্রয়োজন না থাকা সত্ত্বেও reference alive থাকলে memory unnecessarily retained হতে পারে।

Example concept:

```
function createHandler() {  
  const hugeData = new Array(1000000).fill("data");  
  
  return function () {  
    console.log(hugeData.length);  
  };  
}
```

Returned function alive থাকলে `hugeData` -ও প্রয়োজনীয় হতে পারে বলে retained থাকবে।

তাই long-lived closures-এর ক্ষেত্রে memory usage সম্পর্কে সচেতন থাকা ভালো।

12. Closures in Event Handlers

DOM event handler-এ closure খুব common।

```
function createButtonHandler(userName) {
  return function () {
    console.log(`Hello ${userName}`);
  };
}

const handler = createButtonHandler("Shariar");

button.addEventListener("click", handler);
```

`handler` তার outer `userName` remember করে।

13. Closures in Timers

```
function countdown(start) {
  let count = start;

  const timer = setInterval(() => {
    console.log(count);
    count--;

    if (count < 0) {
      clearInterval(timer);
    }
  }, 1000);
}

countdown(3);
```

Callback:

```
() => {
  console.log(count);
}
```

`count` -কে closure-এর মাধ্যমে access করছে।

14. Closures in React

React শেখার সময় closure অনেক গুরুত্বপূর্ণ।

Example:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    console.log(count);  
  }  
  
  return <button onClick={handleClick}>Click</button>;  
}
```

`handleClick` function তার render-এর lexical environment থেকে `count` access করে।

React-এ render-এর সাথে closure-এর relationship বুঝলে পরে এই বিষয়গুলো বুঝতে সহজ হয়:

- stale closures
- `useEffect`
- dependency array
- event handlers
- `useCallback`
- asynchronous callbacks

15. Stale Closure — Preview

এটা Intermediate JavaScript + React-এর খুব important concept।

```
let count = 0;  
  
function logCount() {  
  console.log(count);  
}  
  
count = 10;  
  
logCount();
```

Output:

10

কারণ এখানে variable binding-এর current value access হচ্ছে।

কিন্তু asynchronous code এবং React render-এর ক্ষেত্রে closure পুরোনো lexical environment ধরে রাখতে পারে।

Example:

```
function createLogger(value) {  
  return function () {  
    console.log(value);  
  };  
}  
  
const logger = createLogger(10);  
  
logger();
```

পরে অন্য variable change করলেও:

```
let value = 20;
```

logger-এর captured `value` magically change হবে না।

16. Closure-এর সবচেয়ে গুরুত্বপূর্ণ Properties

Concept	Meaning
Lexical Scope	Code কোথায় লেখা হয়েছে তার উপর scope নির্ভর করে
Scope Chain	Variable খুঁজতে outer scopes-এ যাওয়া
Closure	Function + remembered lexical environment
Private State	Closure দিয়ে data বাইরে থেকে hide করা
Factory	Closure দিয়ে customized function তৈরি
Memoization	Closure-এর cache ব্যবহার করে result reuse
Partial Application	কিছু argument আগে থেকে fixed করা
Shadowing	Inner variable outer variable-কে hide করা

17. Closure বনাম Scope

দুটোকে গুলিয়ে ফেলবে না।

Scope

কোন variable কোথায় accessible?

```
function test() {  
  const x = 10;  
}
```

☒ শুধু function scope-এর মধ্যে accessible।

Closure

একটা function তার outer lexical environment-এর variable মনে রাখে।

```
function outer() {  
  const x = 10;  
  
  return function () {  
    console.log(x);  
  };  
}
```

এখানে returned function ☒ retain করে।

18. Closure বনাম Global Variable

Bad approach:

```
let count = 0;  
  
function increment() {  
  count++;  
}
```

যেকোনো code:

```
count = 999;
```

করতে পারে।

Closure approach:

```
function createCounter() {  
  let count = 0;  
  
  return {  
    increment() {  
      count++;  
    },  
  
    getCount() {  
      return count;  
    }  
  };  
}
```

এখানে state encapsulated।

19. Common Mistakes

Mistake 1 — Closure মানেই শুধু returned function

না।

Callback, event handler, timer—অনেক জায়গায় closure তৈরি হয়।

Mistake 2 — Closure variable-এর copy রাখে

Closure সাধারণত variable binding/environment-এর সাথে কাজ করে; শুধু primitive value-এর একটা আলাদা copy ধরে রাখে—এভাবে ভাবা ঠিক নয়।

Example:

```
function counter() {  
  let count = 0;
```

```
return function () {  
  count++;  
  return count;  
};  
}
```

প্রতিবার call-এ একই retained `count` binding update হচ্ছে।

Mistake 3 — Shadowing মানে value change

না।

```
let x = 10;  
  
{  
  let x = 20;  
}
```

এখানে outer `x` এখনও `10`।

Mistake 4 — সব closure memory leak

না।

Closure JavaScript-এর normal এবং fundamental feature।

Problem হয় যখন unnecessary long-lived references রাখা হয়।

Mistake 5 — `var` loop behavior না বোঝা

এই pattern interview-এ খুব common:

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Output:

3
3
3

এর মূল কারণ:

function scope + closure + asynchronous callback timing

20. Practical Challenge 1 — Counter

একটি function তৈরি করো:

```
const counter = createCounter();  
  
counter.increment();  
counter.increment();  
  
console.log(counter.getCount());
```

Expected:

2

Requirements:

- `count` private হতে হবে
- বাইরে থেকে `count` access করা যাবে না
- `increment()`
- `decrement()`
- `getCount()`

21. Practical Challenge 2 — Memoization

এমন একটি:

```
memoize(fn)
```

function বানাও যেটা:

```
const slowAdd = memoize((a, b) => {  
  console.log("Calculating...");  
  return a + b;  
});
```

এরপর:

```
slowAdd(10, 20);  
slowAdd(10, 20);
```

দ্বিতীয়বার যেন "Calculating..." print না হয়।

22. Practical Challenge 3 — Discount Factory

এমন function বানাও:

```
const tenPercentOff = createDiscount(0.10);  
const twentyPercentOff = createDiscount(0.20);
```

তারপর:

```
tenPercentOff(1000);  
twentyPercentOff(1000);
```

Expected:

```
900  
800
```

23. Output Prediction

Question 1

```
function outer() {  
  let x = 10;
```

```
    return function () {  
      x++;  
      console.log(x);  
    };  
  }  
  
  const fn = outer();  
  
  fn();  
  fn();
```

Output কী?

<details> <summary>Answer</summary>

```
11  
12
```

কারণ দুইবার call একই closure-এর ☒ ব্যবহার করছে।

</details>

Question 2

```
let x = "Global";  
  
function test() {  
  let x = "Local";  
  
  return function () {  
    console.log(x);  
  };  
}  
  
const fn = test();  
  
fn();
```

Output কী?

<details> <summary>Answer</summary>

Local

কারণ Local variable global x-কে shadow করছে।

</details>

Question 3

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Output:

```
3  
3  
3
```

Question 4

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Output:

```
0  
1  
2
```

24. Interview Questions

Beginner/Intermediate

1. Closure কী?
2. Lexical scope কী?

3. Scope chain কী?
4. Closure কীভাবে তৈরি হয়?
5. Function execution শেষ হওয়ার পরও variable কীভাবে accessible থাকে?
6. Closure-এর practical use cases কী?
7. Closure দিয়ে private variable কীভাবে তৈরি করবে?
8. Factory function কী?
9. Memoization কী?
10. Memoization-এর জন্য closure কেন useful?
11. Partial application কী?
12. Currying আর partial application-এর difference কী?
13. Variable shadowing কী?
14. Shadowing এবং reassignment-এর difference কী?
15. `var` এবং `let` loop-এর closure behavior কেন আলাদা?
16. Closure কি memory leak তৈরি করে?
17. Event handler-এর সাথে closure-এর সম্পর্ক কী?
18. Callback function-এ closure কোথায় দেখা যায়?
19. React-এ closure কেন important?
20. Stale closure কী?

25. Quick Revision

Lexical Scope

↓

Function কোথায় লেখা হয়েছে

↓

Scope Chain

↓

Outer variables access

↓

Closure

↓

Function + remembered lexical environment

Closure ব্যবহার করে:

Private State

↓

Factory Functions

↓

Memoization

↓

Partial Application

↓

Callbacks



Event Handlers



Timers



React Patterns

26. সবচেয়ে গুরুত্বপূর্ণ Code Pattern

এই patternটা ভালোভাবে মুখস্থ না করে বুঝে ফেলো:

```
function outer() {  
  let state = 0;  
  
  return function inner() {  
    state++;  
  
    return state;  
  };  
}  
  
const fn = outer();  
  
console.log(fn());  
console.log(fn());  
console.log(fn());
```

Output:

```
1  
2  
3
```

কারণ:

```
fn  
|  
├─ function inner  
|
```

```
└─ closure
  └─ state = 0
```

প্রতিবার:

```
fn() → state = 1
fn() → state = 2
fn() → state = 3
```

27. Final Mental Model

Closure-কে এইভাবে মনে রাখো:

A closure is a function that remembers the variables from the lexical scope where it was created.

আরও সহজ করে:

```
Function তৈরি হলো
  ↓
তার surrounding scope মনে রাখলো
  ↓
Function অন্য জায়গায় গেল
  ↓
Outer function শেষ হয়ে গেল
  ↓
তবুও remembered variables accessible থাকলো
  ↓
এটাই Closure
```

সবচেয়ে পুরুত্বপূর্ণ তিনটি জিনিস:

```
Lexical Scope
+
Function
+
Remembered Environment
=
Closure
```

আর Closure-এর উপর ভালো দখল থাকলে পরের দিকে:

Callbacks
Promises
Async JavaScript
Event Loop
React Hooks
useEffect
useCallback
useMemo
Stale Closures
Functional Programming

এসব concept অনেক সহজে বোঝা যাবে।

Mastery Checklist

- ☐ Scope বুঝি
- ☐ Lexical Scope বুঝি
- ☐ Scope Chain বুঝি
- ☐ Closure নিজের ভাষায় explain করতে পারি
- ☐ Closure দিয়ে private state বানাতে পারি
- ☐ Factory function বানাতে পারি
- ☐ Memoization implement করতে পারি
- ☐ Partial application বুঝি
- ☐ Currying এবং partial application আলাদা করতে পারি
- ☐ Variable shadowing বুঝি
- ☐ `var` বনাম `let` closure behavior বুঝি
- ☐ Callback-এর মধ্যে closure identify করতে পারি
- ☐ Timer-এর মধ্যে closure identify করতে পারি
- ☐ React-এ closure কেন গুরুত্বপূর্ণ তার basic idea আছে
- ☐ Stale closure-এর concept বুঝতে শুরু করেছি